ETL/ELT Architecture Design with AWS & ML/LLM Integration

Overview

Modern data pipeline architecture supporting both ETL (Extract-Transform-Load) and ELT (Extract-Load-Transform) patterns, optimized for AWS cloud infrastructure with ML/LLM capabilities.

Data Flow Patterns

Pattern 1: Batch ETL with ML

```
Source DB → DMS → S3 Raw → Glue ETL → S3 Curated →
  ↓
  → SageMaker Feature Store → Model Training → Predictions
```

Pattern 2: Streaming ELT with LLM

```
Kinesis Stream → Firehose → S3 Raw →
  ↓
  → Athena Transforms → Curated Layer →
  ↓
  → Lambda + Bedrock → Enriched Data → Applications
```

Pattern 3: Hybrid (ETL + ELT)

```
Multiple Sources → Glue/Lambda → S3 Raw →
  ↓ (ETL)                        ↓ (ELT)
  Redshift ← Glue Transform      Athena Queries
  ↓                              ↓
  BI Tools                       ML Pipelines
```

Key AWS Services Summary

Component	AWS Service	Purpose
Ingestion	Glue, Kinesis, DMS, Lambda	Data extraction
Storage	S3, Redshift	Raw & curated data
Transform	Glue, EMR, Athena, Lambda	Data processing
Orchestration	Step Functions, EventBridge, MWAA	Workflow management
ML/LLM	SageMaker, Bedrock	Model training & inference
Governance	Lake Formation, Glue Catalog	Metadata & security
Monitoring	CloudWatch, X-Ray	Observability

ML/LLM-Specific Considerations

Feature Engineering

- Use Glue/EMR for large-scale feature computation
- SageMaker Processing for complex transformations
- Feature Store for serving and versioning

LLM Data Preparation

- **Prompt Engineering**: Store prompt templates in S3
- **Context Windows**: Chunk large documents (Lambda/Glue)
- **RAG Architecture**: Vector embeddings in OpenSearch/RDS pgvector
- **Fine-tuning Data**: Curated datasets in S3 for model customization

Model Serving

- **Batch Predictions**: Glue/EMR → SageMaker Batch Transform
- **Real-time**: API Gateway → Lambda → SageMaker/Bedrock
- **Streaming**: Kinesis → Lambda → Feature Store → Inference

---

Data Governance

Security

- **IAM Roles**: Least privilege access
- **KMS Encryption**: At-rest and in-transit
- **VPC Endpoints**: Private connectivity
- **Secrets Manager**: Credentials management

Quality

- **Glue Data Quality**: Automated validation rules
- **Great Expectations**: Custom quality checks
- **LLM-based Validation**: Bedrock for semantic checks

Lineage

- **Glue Catalog**: Table-level lineage
- **Tags**: Business context and ownership
- **CloudTrail**: Audit logs

Orchestration Examples

ETL Workflow (Step Functions)

```
{
  "Extract": "DMS/Glue Crawler",
  "Transform": "Glue ETL Job",
  "Validate": "Lambda + Data Quality",
  "Load": "Copy to Redshift",
  "Notify": "SNS Alert"
}
```

ML Pipeline (SageMaker Pipelines)

```
{
  "DataPrep": "Glue/Processing Job",
  "FeatureEngineering": "SageMaker Processing",
  "Training": "SageMaker Training",
  "Evaluation": "Lambda + Bedrock (model assessment)",
  "Deployment": "SageMaker Endpoint"
}
```

Cost Optimization

1. **S3 Intelligent Tiering**: Auto-move to cheaper storage classes
2. **Redshift RA3**: Separate compute and storage
3. **Glue Auto Scaling**: Dynamic DPU allocation
4. **Spot Instances**: For EMR clusters
5. **Bedrock On-Demand**: Pay per token vs provisioned throughput
6. **Athena**: Query only required partitions

Monitoring & Observability

- **CloudWatch Dashboards**: Pipeline health metrics
- **CloudWatch Alarms**: Failed jobs, SLA breaches
- **X-Ray**: End-to-end tracing for Lambda/API calls
- **Glue Job Metrics**: DPU usage, record counts
- **SageMaker Model Monitor**: Drift detection
- **SNS/Slack**: Alerting

Example Use Cases

1. Customer 360 (ETL)

```
CRM/ERP/Web → Glue → S3 → Redshift → QuickSight
                ↓
              Bedrock (entity resolution, deduplication)
```

2. Real-time Fraud Detection (Streaming + ML)

```
Transactions → Kinesis → Lambda → Feature Store → SageMaker Endpoint → Alert
```

3. Document Intelligence (LLM)

```
PDFs → S3 → Textract → Lambda + Bedrock → Extract entities → Redshift → Search
```

4. Predictive Analytics (Hybrid)

```
Historical Data → S3 → Athena (ELT) → Feature Store →
                                            ↓
                                    SageMaker Training → Inference
```

Technology Stack

- **Languages**: Python (PySpark, Boto3), SQL
- **IaC**: Terraform or CloudFormation
- **Orchestration**: Step Functions, MWAA (Airflow), EventBridge
- **Version Control**: Git, CodeCommit
- **CI/CD**: CodePipeline, GitHub Actions

Next Steps

1. Define specific use case requirements
2. Design detailed data models (schemas)
3. Implement proof of concept for one pipeline
4. Set up monitoring and alerting
5. Document data dictionary and lineage
6. Establish governance policies
7. Train team on AWS services and ML/LLM tools

This design provides a flexible foundation that can scale from simple batch ETL to complex real-time ML/LLM workflows on AWS.